OpenBCA

This library provides aggregators, program administrators, utilities, and regulators a means to configure and execute Jurisdiction-Specific Tests (JSTs) for demand side programs/portfolios in accordance to guidance in the National Standard Practice Manual.

Accurate cost-effectiveness tests that account for energy impacts and progress toward other policy objectives are critical for optimal demand side program design and informed decision making. However, traditional cost effectiveness tests (e.g., Total Resource Cost test, Utility Cost Test etc.) are often too restrictive, utilize non-transparent inputs, and do not fully reflect the goals and objectives of a jurisdiction. In such cases, benefit-cost testing can lead to poor demand side program design and ultimately unbalanced investment across energy resources. To help address these shortcomings, E4TheFuture's National Energy Screening Project (NESP) published the National Standard Practice Manual for Benefit-Cost Analysis of DERs (NSPM) in 2020. The NSPM provides a set of core principles and a process for developing complete and symmetric JSTs for demand side programs. Following the NSPM guidance, a regulator, utility, and/or other party can develop a JST that properly accounts for the utility system costs and benefits of a DER program or investment strategy, as well as any non-utility system impacts applicable to the jurisdiction's priority policy goals and objectives.

To support a balanced and comprehensive BCA architecture, this library enables comprehenseive and flexible configuration and computation required for the formulation of JSTs. The OpenBCA is designed to be used in one of two main ways:

1. Standalone Pathway

This mode is intended for non-technical users. It is designed to run exclusively using local hardware and operate end-to-end without the need for users to write code, use terminal applications, or manage databases.

Defining characteristics of the Standalone pathway include:

- Use of the Excel input templates
- Use of the user interface to upload completed input templates, launch the model, and explore and download results.

The amount of data that can be processed and computational speed to run the OpenBCA model will be limited by the user's hardware. Generally it is recommended users have at least 16 GB of RAM, though for smaller jobs less will suffice.

2. Integrated Pathway

This mode is intended for inclusion of the OpenBCA in existing sofware systems and technical workflows. For instance, if a user needs to leverage cloud computing or data storage resources

or wishes to automate benefit-cost analysis as part of a larger analytics pipeline. The integrated pathway is most appropriate for users who:

- Need to scale analysis across very large datasets
- Need maximum flexibility for novel use cases
- Need to embed OpenBCA in existing data pipelines and workflows
- Wish to develop custom user interfaces

Users of the integrated pathway will need to input data as defined by the schemas resulting from the input parsing step of the Standalone pathway (and reproduced below for reference).

Generally speaking, the Integrated pathway is for expert users and will not be explicity supported by the development team.

Prerequisite Installations

The OpenBCA requires the following packages to be installed on your local machine: - git (mac or windows) - uv mac or windows - make (Note this is included as standard on Mac machines)

Running OpenBCA

To run the OpenBCA, use the following commands

Launching the UI:

```
make run-openbca
```

or (without make):

```
uv run streamlit run user_interface/Entrypoint.py
```

Through the UI users can upload input files, launch the model, and view results.

Running OpenBCA without the UI:

```
make run-openbca-model
```

or (without make):

```
uv run python model_runners.py openbca_excel_model
```

With this option users can launch the SQLmesh pipeline and generate the output database. This option still begins with the parsing of input files, which need to be stored in the `excel_input_parsing/input_templates` folder.

In either of these options, the output database is stored in the `output/openbca.db` file.

🛠️ Key Dependencies

The OpenBCA software heavily leverages:

- uv for Python package management.
- Pandas ExcelFile for parsing data from Excel input templates.
- SQLmesh to orchestrate data and computational pipelines.
- DuckDB for local database management and execution of SQL queries.
- Streamlit for the base framework of the user interface.

⚙️ Project Architecture

The repository contains three main related systems housed in the following folders:

- `excel_input_parsing/` This directory stores the populated input templates and contains a SQLmesh pipeline to parse the input files into the OpenBCA schema.
- `core/` The heart of the OpenBCA logic. This houses the generic, jurisdiction-agnostic impact computation code, again in the form of a SQLmesh pipeline. It can run with any input backend (CSV, Excel, BigQuery, Streamlit app, etc.). It only generates SQL views to let the client application handle the actual data loading and output.
- `user_interface/` This folder contains a Streamlit application that launches a web app. The web app contains a page that allows users to upload populated input templates and another page devoted to exploration of results.

🧱 BCA Basic Components

Benefit-cost analysis for distributed energy resources is conducted using the following information and data:

Annual Savings - The amount per year that an intervention saves. Savings are tied to specific **commodities**. An intervention can save across multiple commodities and can be negative. For instance, a heat pump electrification measure will decrease natural gas consumption (positive savings), increase electricity consumption (negative savings), and enhance societal resiliance and host-customer reliability. In this example “savings” are generated across four commodities: natural gas, electricity, societal resiliance, and host-customer reliability.

Avoided Costs - Within each commodity, there may be one or more avoided costs. Each avoided cost represents a specific, quantifiable dollar value tied to an effect that can be isolated. For example, electricity savings may avoid energy procurement costs, GHG emissions, various capacity costs and more. The NSPM defines many common utility system and non-system avoided costs that should be accounted for in a JST. Avoided costs should

represent **marginal** values. For instance, if a program saves 1 MWh, the avoided costs should reflect the dollar value from that specific MWh instead of the average of all electricity generated during the time the savings occured. Marginal values can be higher or lower than average values depending on the context. If a program saves a MWh during a peak period, then that savings will direclty reduce reliance on an expensive peaker plant. In contrast, if that MWh were saved during a period of renewable curtailment, the dollar value may be zero or even negative.

Savings Load Shapes - Encode the distribution of savigns over time. The OpenBCA supports natural gas and electric savings load shapes of annual, monthly, daily, or hourly granularities. Other commodities are limited to annual values. The *maximum* granularity of avoided cost profiles per commodity establishes the *minimum* granularity that a savings load shape must meet. Savings load shapes can be entered as dimensioned or normalized values. If normalized the load shape acts to distribute annual savings across the year. If dimensioned, the load shape is expected to sum to an annual savings value and a value of 1 should be entered for the corresponding annual savings for that commodity.

Discount Rate and Cadence - The calculation of benefits and costs is conducted via a net-present-value (NPV) computation over the lifecycle of impacts from an intervention. The annual discount rate determines the degree to which future benefits are eroded relative to the opportunity cost of the capital invested in the project. The discount cadence determines how many time periods will be included in the NPV calculation. Currently the OpenBCA supports annual and quarterly discounting.

Inflation Rate - Users can choose whether to report results in real or nominal dollars. If real dollars are desired then the user can enter a base year and inflation rate to adjust dollars to the base year. The base year can be before, during, or after a program's impacts.

Cost Treatment - Determines whether certain quantities are treated as costs or transfer payments within a JST. If the user selects "TRC" then incentives paid from utilities to customers are treated as transfer payments and are effectively eliminated from the calculation. If "UCT" is selected then incentives are treated as costs. Similarly, host customer tax incentives act to reduce total costs in a TRC framework, but are ignored in a PAC framework. In general, if host customer benefits are to be included in a test then it is recommended to use the TRC framework. If only the utility's perspective is the subject of a JST then the UCT framework is recommended and the user should take care to not include host customer benefits in the configuration of the JST.

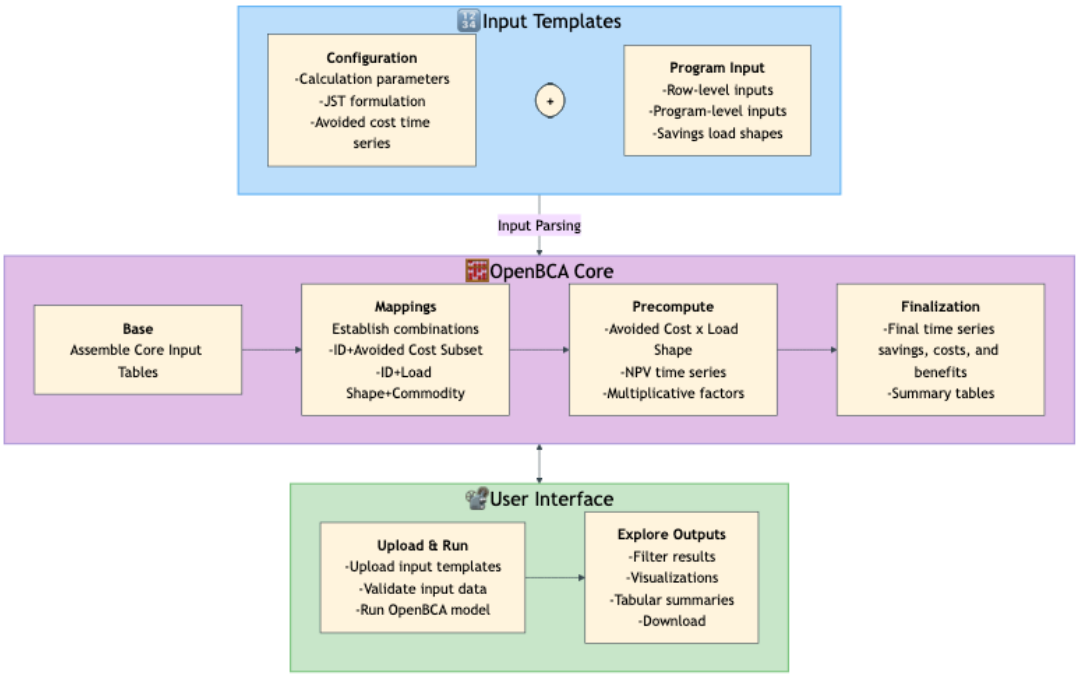
Expected Useful Life (EUL) - The number of years that a project is expected to deliver impacts.

Net-to-Gross (NTG) - Intended to account for free ridership, this metric is used in some jurisdicitons to represent the fraction of program benefits and host customer costs that occured *because of the program*. NTG values typically range between 0 and 1 and 1 - NTG is interpreted as freeridership, the fraction of program impacts tied to customers who would have undertaken the interventions even in the abscence of the program. NTG values above 1.0 are

allowed as some jurisdictions will assume some benefits occur outside the program *but on account of the program*. This is referred to as “spillover” or “market effects.”

❯❯❯ OpenBCA Process Flow

This diagram shows the execution flow of the OpenBCA across three main phases: data input, computation, and user interface functionality

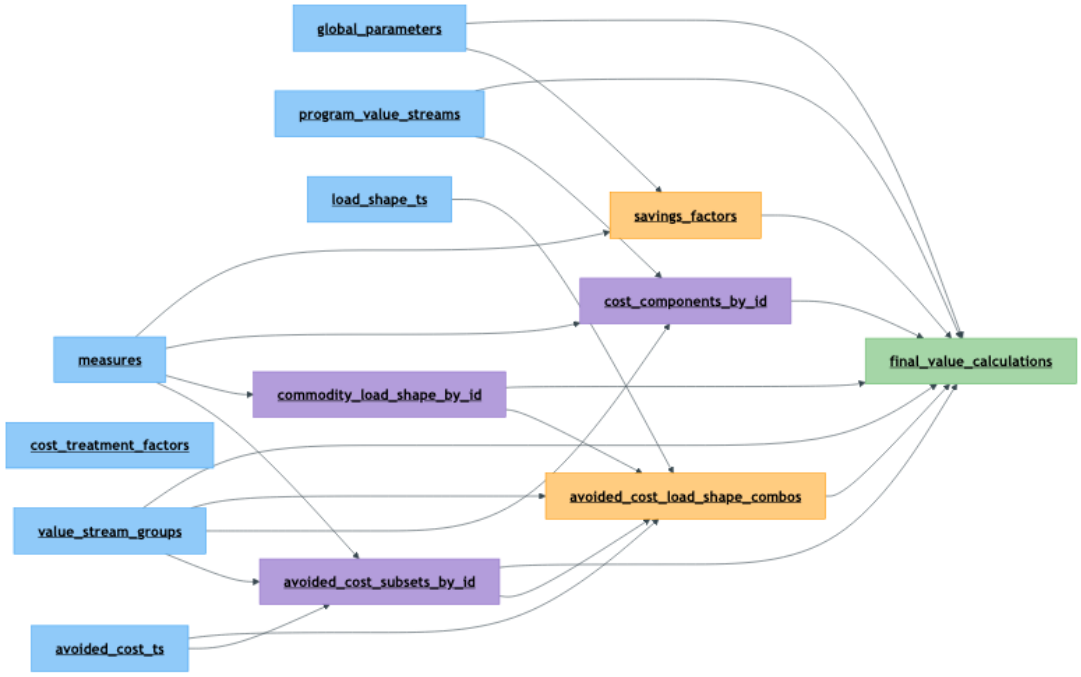


Mermaid diagram

OpenBCA Core Model

This diagram shows the flow of the OpenBCA core model across its four phases:

Mermaid diagram



Mermaid diagram

Core Model Table Reference

This table lists the tables generated during core model execution,* along with their key contents:

Layer	Table	Contents
Base	**measures**	Row-level inputs: unique ID, metadata, EUL, NTG, annual savings and costs, load shape assignments
Base	**global_parameters**	Dollar year, NPV parameters, symmetry treatment, line loss factors
Base	**cost_treatment_factors**	Establishes cost and benefit multipliers for specific test frameworks (TRC, UCT etc.)
Base	**program_value_streams**	Program-level costs and benefits by year
Base	**value_stream_groups**	Info to shepherd each value stream into a specific computational treatment
Base	**load_shape_ts**	Savings load shapes time series
Base	**avoided_cost_ts**	Avoided costs time series
Mapping	**avoided_cost_subsets_by_id**	Mapping between ID, avoided cost, and avoided cost subset
Mapping	**commodity_load_shape_by_id**	Mapping between ID, commodity, and load shape
Mapping	**cost_components_by_id**	

Layer	Table	Contents
		Establishes costs and multiplicative factors by ID for row-level inputs and by program name for program-level inputs
Precompute	**avoided_cost_load_shape_combos**	Calculates avoided cost x savings load shape across the EUL for all necessary combinations of avoided cost, savings load shape and year
Precompute	**savings_factors**	Calculates discount and inflation factors across the full EUL for every row-level input. Applies those factors along with unit quantity, NTG, EUL, line losses as appropriate for every commodity
Finalization	**final_value_calculations**	Combines the precomputed values from *avoided_cost_load_shape_combos*, *savings_factors*, and *cost_components_by_id*, along with information from several other tables, into full time-series vectors of final net savings and value streams for benefits and costs

*Note that a few additional tables are generated as part of the Finalization step but are not listed here.

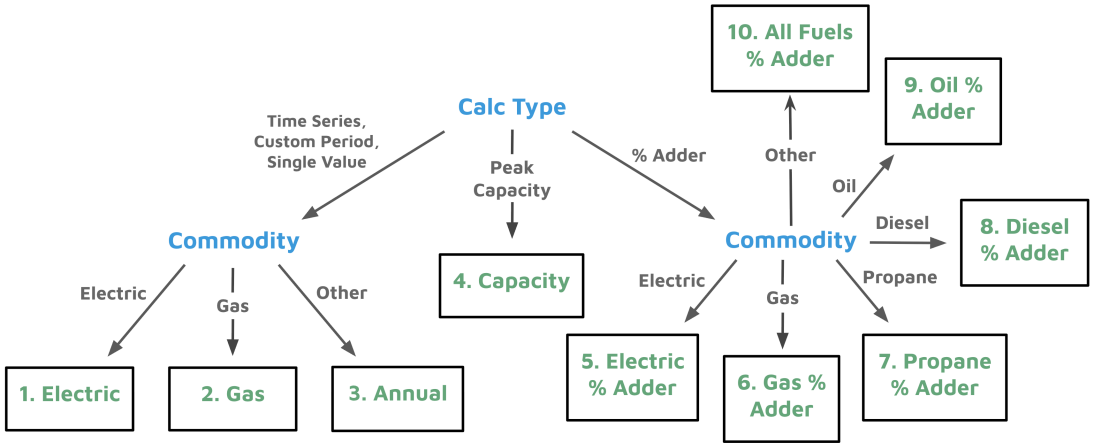
📱 Value Stream Calculation

There are ten computational pathways supported by the OpenBCA to properly handle different types of value streams. The flow diagram and equation reference below provide details on the logic and mathematics.

To determine which pathway a value stream will follow the OpenBCA first checks the calculation type. In the Standalone pathway this is a required field entered in the Configuration input template for each value stream. If the calculation type is some form of time series (including custom period or single value), then the commodity is referenced and the pathway is assigned as Electric, Natural Gas, or Annual accordingly.

If the calculation type is Capacity then the corresponding pathway is assigned.

Finally, if the calculation type is % Adder, then Commodity is again checked and the assignment is made accordingly.



Value stream groups

Equations for the calulcation of benefits and costs tied to each pathway are given below.

$$F = NTG \times Unit\ Quantity$$

$$D = 1/(1 + (Discount\ Rate - Inflation\ Rate)/4)^{((Y-SY)\times 4+SQ-1)}$$ (Quarterly Discounting)

$$D = 1/(1 + (Discount\ Rate - Inflation\ Rate))^{(Y-SY)}$$ (Annual Discounting)

$$I = 1/(1 + Inflation\ Rate)^{(Y-DY)}$$

$$L = 1/(1 - Line\ Loss)$$

1. **Electric**

$$\$\ Benefits = \sum_{y=SY}^{SY+EUL} \sum_{t=1}^{N} F \times D \times I \times L_E \times Avoided\ Cost_{y,t} \times Annual\ Savings \times Load\ Shape_t$$

2. **Gas**

$$\$\ Benefits = \sum_{y=SY}^{SY+EUL} \sum_{t=1}^{N} F \times D \times I \times L_G \times Avoided\ Cost_{y,t} \times Annual\ Savings \times Load\ Shape_t$$

3. **Annual**

$$\$\ Benefits\ or\ \$Costs = \sum_{y=SY}^{SY+EUL} F \times D \times I \times Avoided\ Cost_y \times Annual\ Savings$$

4. **Capacity**

$$\$\ Benefits = \sum_{y=SY}^{SY+EUL} F \times D \times I \times L_P \times Avoided\ Cost_y \times Peak\ Savings$$

5. **Electric % Adder**

$$\$\ Benefits = \$Energy \times \%\ Adder$$

6. **Gas % Adder**

$$\$\ Benefits = \$Fuel\ Cost\ and\ O\&M \times \%\ Adder$$

7. **Propane % Adder**

$$\$\ Benefits = \$Propane\ Supply \times \%\ Adder$$

8. **Diesel % Adder**

$$\$\ Benefits = \$Diesel\ Supply \times \%\ Adder$$

9. **Oil % Adder**

$$\$\ Benefits = \$Oil\ Supply \times \%\ Adder$$

10. **All Fuels % Adder**

$$\$\ Benefits = (\$Energy\ (Elec) + \$Fuel\ Cost\ and\ O\&M\ (NG) + \$Propane\ Supply + \$\ Diesel\ Supply + \$Oil\ Supply) \times \%\ Adder$$

Variable	Definition
NTG	Net-to-Gross ratio
Y	Year
SY	Start Year - the first calendar year an intervention has an impact

Variable	Definition
SQ	Start Quarter - the first quarter an intervention has an impact
DY	Dollar Year - the year to pin the calculation of real dollars when adjusting for inflation
EUL	Expected Useful Life (years)
N	The number of temporal increments in a year for a particular value stream. For example N = 12 for monthly data.
Avoided Cost$_{y,t}$	Marginal avoided cost ($/commodity unit) for year y and time period t. For instance, $/kWh for hour 7354 of 2035
Avoided Cost$_{y}$	Marginal avoided cost ($/commodity unit) for year y. For instance, $/kWh for 2035
Annual Savings	Annual savings (1.0 or commodity unit) for an intervention. For instance, kWh. If the load shape is dimensioned then Annual Savings should be set to 1.0
Load Shpae$_{t}$	Load Shape (commodity unit or fraction) for time period t. For instance, $/kWh for hour 7354 of the year or 0.001 for hour 7354, which assigns 0.1% of the annual savings to that hour.
L_E	Line loss factor using the electric line loss rate
L_G	Line loss factor using the natural gas line loss rate
L_P	Line loss factor using the peak period electric line loss rate (used only in the Capacity pathway)

% Adders

Several of the value stream groups are % Adders. These value streams are intended to provide users mechanisms to acknowledge and include benefits from value streams that have not yet been properly modeled. For example, consider a jurisdiction that establishes enhancing energy system resilience as a main objective of its demand side portfolio. However, modeling yielding detailed avoided cost data may be a future endeavor. Instead of leaving the value stream out, parties could agree that a 5% adder should be included until more definitive modeling outputs are available.

It is important to note that for each % Adder value stream, there is a more fundamenal value stream upon which the % Adder calculation is based. The table below makes explicit the % Adder value streams and the associated value stream(s) that serve as the basis for calculation.

% Adder Value Stream Group	Basis Value Stream
Electric	Energy
Natural Gas	Fuel Cost and O&M
Propane	Propane Supply
Diesel	Diesel Supply
Oil	Oil Supply
All Fuels	Energy + Fuel Cost and O&M + Propane Supply + Diesel Supply + Oil Supply

Avoided Cost Subsets

For some or many value streams, avoided costs may be different depending on region, population, or other factors. For instance, if a utility's grid has a segment with more frequent and expensive periods of constraint, then several value streams would be more accurately calculated if:

1. Avoided cost data were grouped into "constrained" and "non-constrained" subsets.
2. Project tracking indicated in which of these regions an intervention occured.

With its Avoided Cost Subset feature, the OpenBCA enables users to enter distinct avoided cost data by divisions they define.

This functionality operates as follows:

For any value stream with defined subsets, distinct avoided cost data can be tied to the specific subset (see the *avoided_cost_ts* table spec below). Then, the user can assign any project to a subset via the *measures* table.

Not all avoided costs need to be assigned to a subset. For example, a user may have different subsets for avoided gnergy costs but only system-wide values for avoided RPS compliance costs. In this case, the user should enter only "System-wide" avoided cost data for the latter. The OpenBCA will automatically probe for and assign projects to specific subsets for the avoided costs that have those subsets, and to a default "System-wide" subset for the remaining avoided costs.

📜 Data Granularity

Given different temporal granularity in avoided cost and savings data - both between jurisdictions and among different value streams even within a single jurisdiction - the OpenBCA input templates and model have been designed to handle natural gas and electric data of the following intervals:

- Hourly

- Daily
- Monthly
- Yearly

Other commodities (delivered fuels, host-customer benefits, costs, and custom commodities) must be yearly inputs.

Within the natural gas and electric commodities, individual avoided cost streams may have different granularities. For example, a jurisdiction may have hourly marginal electric avoided costs compiled for energy procurement, montly for GHG emissions, and annual for RPS compliance.

The OpenBCA handles each case, retaining the granularity of the original avoided cost data throughout.

However, the OpenBCA will not convert lower-granularity savings data to match higher granularity avoided cost data. Therefore, the highest granularity avoided cost stream will determine the savings load shape granularity expected by the software. In other words, the OpenBCA will fail if, for example, hourly avoided cost data are provided for one or more value streams, but monthly savings load shapes are input.

Alignment of Load Shapes and Avoided Costs

The OpenBCA allows users to enter savings load shapes for one year time periods, amounting to 8760 hourly data points, 365 daily data points, 12 monthly data points, or 1 annual data point. The savings load shape is expected to persist across the full Expected Useful Life (EUL) of the intervention.

However, because energy system and non-system marginal avoided costs are expected to evolve over time, distinct avoided cost data must be entered for each year of the EUL.

With only one year of data, the savings load shapes will inherently dictate what day of week corresponds to the days throughout the year - and each year the avoided cost data should align with the resulting calendar.

In addition to these considerations, we note that **currently leap years and daylight savings time are not supported.**

Therefore, it is the user's responsibility to:

- Understand what day of week corresponds to Jan 1 for load shape data.
- Ensure that all load shapes share the same Jan 1 day of week.
- Align *all years* of avoided cost data with the load shape data.
- Properly handle avoided cost data to eliminate leap days. (This may be best accomplished by converting Feb. 29 to Mar. 1 and dropping Dec. 31.)
- Ensure all load shape and avoided cost data are in standard time.

✍🏻 Input Schema and Requirements

measures

measures
A-Z id
A-Z program_name
A-Z measure_id
A-Z project_id
A-Z measure_name
A-Z avoided_cost_subset
123 start_year
123 start_quarter
123 discount_rate
A-Z measure_unit
123 unit_quantity
123 estimated_useful_life
123 ntg
123 administration_costs_upfront_dollar
123 administration_costs_annual_dollar_per_year
123 utility_incentive_upfront_dollar
123 utility_incentive_annual_dollar_per_year
123 incremental_costs_upfront_dollar
123 incremental_costs_annual_dollar_per_year
123 host_customer_transaction_costs_dollar
123 host_customer_interconnection_costs_dollar
123 host_customer_tax_incentive_upfront_dollar
A-Z electric_savings_load_shape
123 annual_electric_savings_kwh
123 coincident_peak_savings_kw
A-Z natural_gas_savings_load_shape
123 annual_natural_gas_savings_mmbtu
123 annual_propane_savings_mmbtu
123 annual_oil_savings_mmbtu
123 annual_diesel_savings_mmbtu
123 host_customer_non_energy_impacts_dollar
123 host_customer_non_energy_impacts_low_income_dollar
123 change_in_host_customer_risk_dollar
123 change_in_host_customer_reliability_dollar
123 change_in_host_customer_resilience_dollar
123 change_in_societal_resilience_dollar
A-Z custom_1_value_stream_name
A-Z custom_1_value_stream_commodity
123 custom_1_annual_savings
A-Z custom_2_value_stream_name
A-Z custom_2_value_stream_commodity
123 custom_2_annual_savings
A-Z custom_3_value_stream_name
A-Z custom_3_value_stream_commodity
123 custom_3_annual_savings
A-Z custom_4_value_stream_name
A-Z custom_4_value_stream_commodity
123 custom_4_annual_savings
A-Z custom_5_value_stream_name
A-Z custom_5_value_stream_commodity
123 custom_5_annual_savings
A-Z label_1
A-Z label_2
A-Z label_3
A-Z label_4
A-Z label_5

program_value_streams

program_value_streams
A-Z program_name
123 program_year
A-Z avoided_cost
123 avoided_cost_value

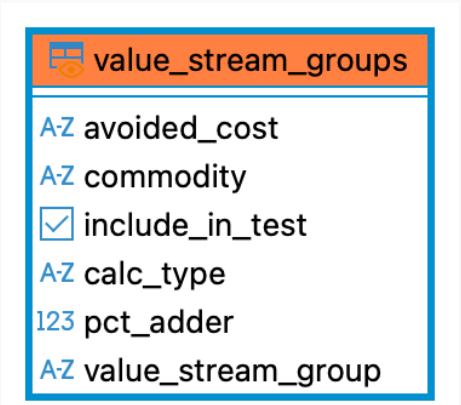
global_parameters

global_parameters
123 inflation_rate
123 dollar_year
123 discount_rate
123 discount_cadence
123 electric_line_loss
123 peak_capacity_line_loss
123 natural_gas_line_loss
A-Z cost_treatment

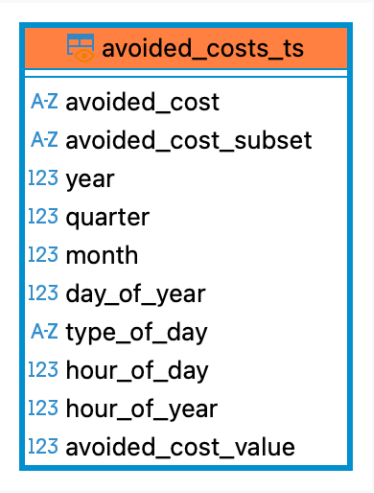
load_shapes_ts

load_shapes_ts
A-Z commodity
123 quarter
123 month
123 day_of_year
123 hour_of_year
A-Z load_shape
123 load_shape_value

value_stream_groups



avoided_costs_ts



This table details requirements for columns in the OpenBCA schema. Not all columns are inclused as many do not have specific requirements beyond data type.

Table	Column	Requirements
measures	**id**	required field - must be unique
measures	**avoided_cost_subset**	must be null or an exact match to an avoided avoided_cost_subset in the avoided_costs_ts table
measures	**start_year**	required field
measures	**start_quarter**	required field - can set to 1 if all value streams are annual
measures	**unit_quantity**	required field
measures	**estimated_useful_life**	required field
measures	**ntg**	required field
measures	**electric_savings_load_shape**	must match a load shape provided in the load_shapes_ts table for the ELECTRIC commodity
measures	**natural_gas_savings_load_shape**	must match a load shape provided in the load_shapes_ts table for the NATURAL GAS commodity
program_value_streams	**program_name**	recommend matchihng to the program_name field of the measures table
global_parameters	**inflation_rate**	requited field - should be set to 0 if nominal dollar outputs are desired
global_parameters	**dollar_year**	

Table	Column	Requirements
		required field - can be set to any integer if using nominal outputs
global_parameters	**discount_rate**	required field unless discount_rate is entered for all rows in the measures table (the measaures discount_rate will be used wherever populated)
global_parameters	**discount_cadence**	required field - accepted values are 1 for annual discounting or 4 for quarterly discounting
global_parameters	**electric_line_loss**	required field if electric value streams are included
global_parameters	**peak_capacity_line_loss**	required field if peak capacity value streams are included
global_parameters	**natural_gas_line_loss**	required field if natural gas value streams are included
global_parameters	**cost_treatment**	required field - accepted values include ‘UCT’, ‘TRC’, and ‘CA TRC’
value_stream_groups	**avoided_cost**	required field
value_stream_groups	**commodity**	required field
value_stream_groups	**include_in_test**	required field
value_stream_groups	**calc_type**	required field - accepted values include ‘electric’, ‘natural_gas’, ‘annual’, ‘capacity’, ‘electric_%_adder’, ‘natural_gas_%_adder’, ‘propane_%_adder’, ‘diesel_%_adder’, ‘oil_%_adder’, ’all_fuels_%_adder’
value_stream_groups	**pct_adder**	required field for any % adder value streams
value_stream_groups	**value_stream_group**	required field
avoided_costs_ts	**avoided_cost**	required field
avoided_costs_ts	**year**	required field
avoided_costs_ts	**month**	required field if corresponding avoided_cost temporal granularity is monthly

Table	Column	Requirements
avoided_costs_ts	**day_of_year**	required field if corresponding avoided_cost temporal granularity is daily
avoided_costs_ts	**hour_of_day**	required field if corresponding avoided_cost temporal granularity is hourly
avoided_costs_ts	**avoided_cost_value**	required field
load_shapes_ts	**commodity**	required field
load_shapes_ts	**month**	required field if corresponding load_shape temporal granularity is hourly
load_shapes_ts	**day_of_year**	required field if corresponding load_shape temporal granularity is daily
load_shapes_ts	**hour_of_day**	required field if corresponding load_shape temporal granularity is hourly
load_shapes_ts	**load_shape**	required field
load_shapes_ts	**load_shape_value**	required field

📦 Packaging the OpenBCA App

If you want to use OpenBCA as a “double-click” executable, you can run `make build-pyinstaller-package` in your terminal. This will package the OpenBCA app into a single directory located at dist/openbca-app.

Inside the dist/openbca-app directory, there will be an “_internal file” and an executable named “openbca-app”. OpenBCA can be run simply by double-clicking the openbca-app executable, which should open a browser window automatically.

Packaging the OpenBCA app allows for easy distribution between machines without the need to clone the code repository. The openbca-app folder can be zipped and transferred to another machine. Both the “_internal” folder and “openbca-app” are required to ensure proper functionality. **Applications built this way will only work on machines with the same operating system the app was built on**, meaning an application built on Windows will only work on Windows machines (not Macs).

If double-clicking does not work, you can run `make run-pyinstaller-package` in the terminal to launch openbca.

Please note that you must click the “Exit OpenBCA” button on the webpage in order to properly shut down the application. If you do not do so, you will have to manually kill the application through task manager (or its equivalent on non-windows platforms).

This packaging is done through PyInstaller, with the specified openbca-app.spec file.

[Optional] Set up DBeaver to connect to the local DuckDB database

If you want to use DBeaver to connect to the local DuckDB database, you can follow these steps:

1. Install DBeaver from dbeaver.io.
2. Open DBeaver and create a new connection.
3. Select “DuckDB” as the database type.
4. In the connection settings, set the database path to `<project base full path>/open-bca/output/openbca.db`. Note you need to first run the `make run-demo` command to create the database file.
5. Click “Test Connection” to ensure the connection is successful.
6. Click “Finish” to create the connection.
7. You can now explore the database schema and run SQL queries against the OpenBCA tables and views.